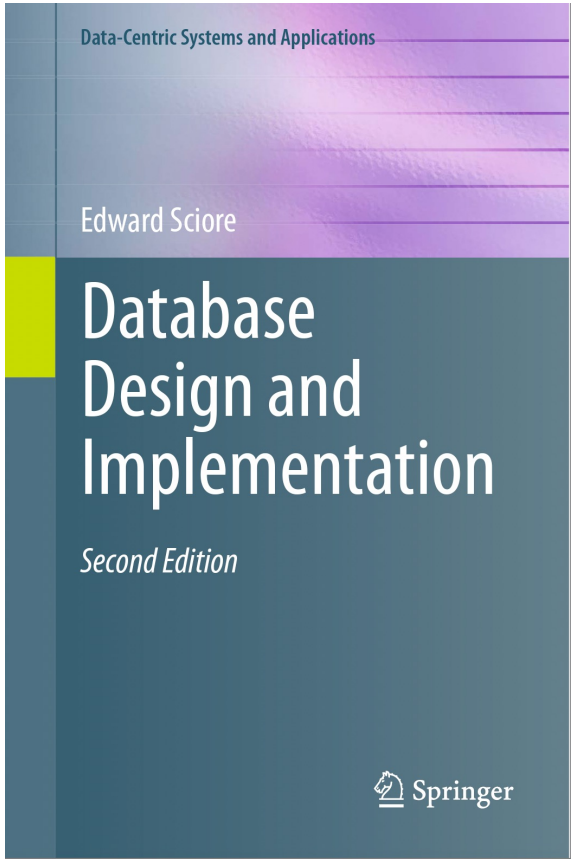
SimpleDB, parte 5

Luigi Bellomarini
luigi.bellomarini@uniroma3.it



SimpleDB, componenti "read-only"

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

Record: gestisce i blocchi per i record delle tabelle

Transaction: gestisce la concorrenza e l'affidabilità

Buffer: mantiene in memoria le pagine per limitare gli accessi

Log: tiene traccia delle operazioni, per l'affidabilità

File: legge e scrive le pagine su disco

Parser

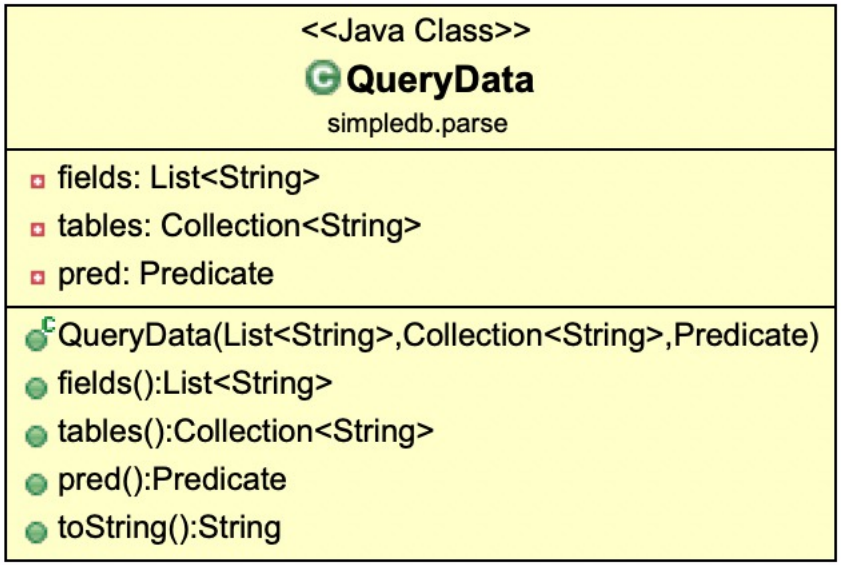
- Un client JDBC invia una query SQL al server sottoforma di stringa;
- Il DBMS processa la stringa creando un oggetto Scan che puo essere eseguito.
- A tale scopo sono necessari due passaggi:

Parser: analisi sintattica

Planner: chiama Parser e determina piano di accesso

- Lo scambio di informazioni tra il Parser ed il Planner avviene tramite un oggetto **QueryData**

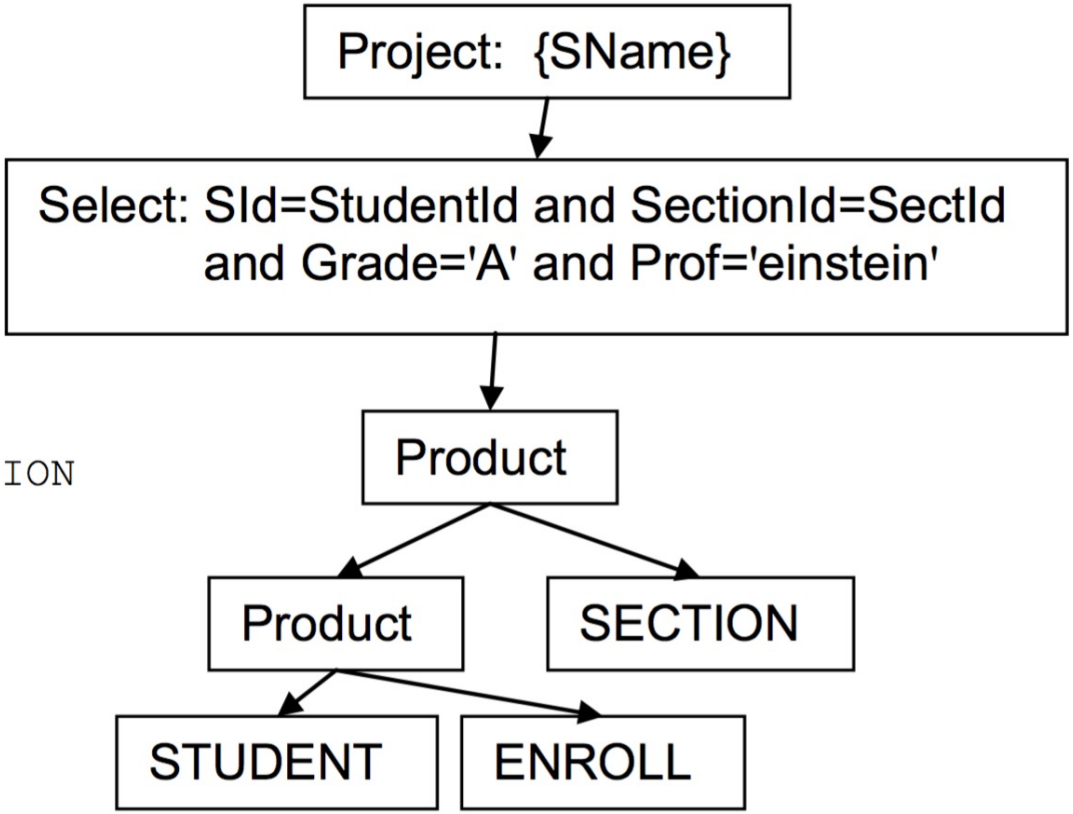
QueryData


<<Java Class>>
QueryData
simpledb.parse
fields: List<String>
tables: Collection<String>
pred: Predicate
QueryData(List<String>,Collection<String>,Predicate)
fields():List<String>
tables():Collection<String>
pred():Predicate
toString():String

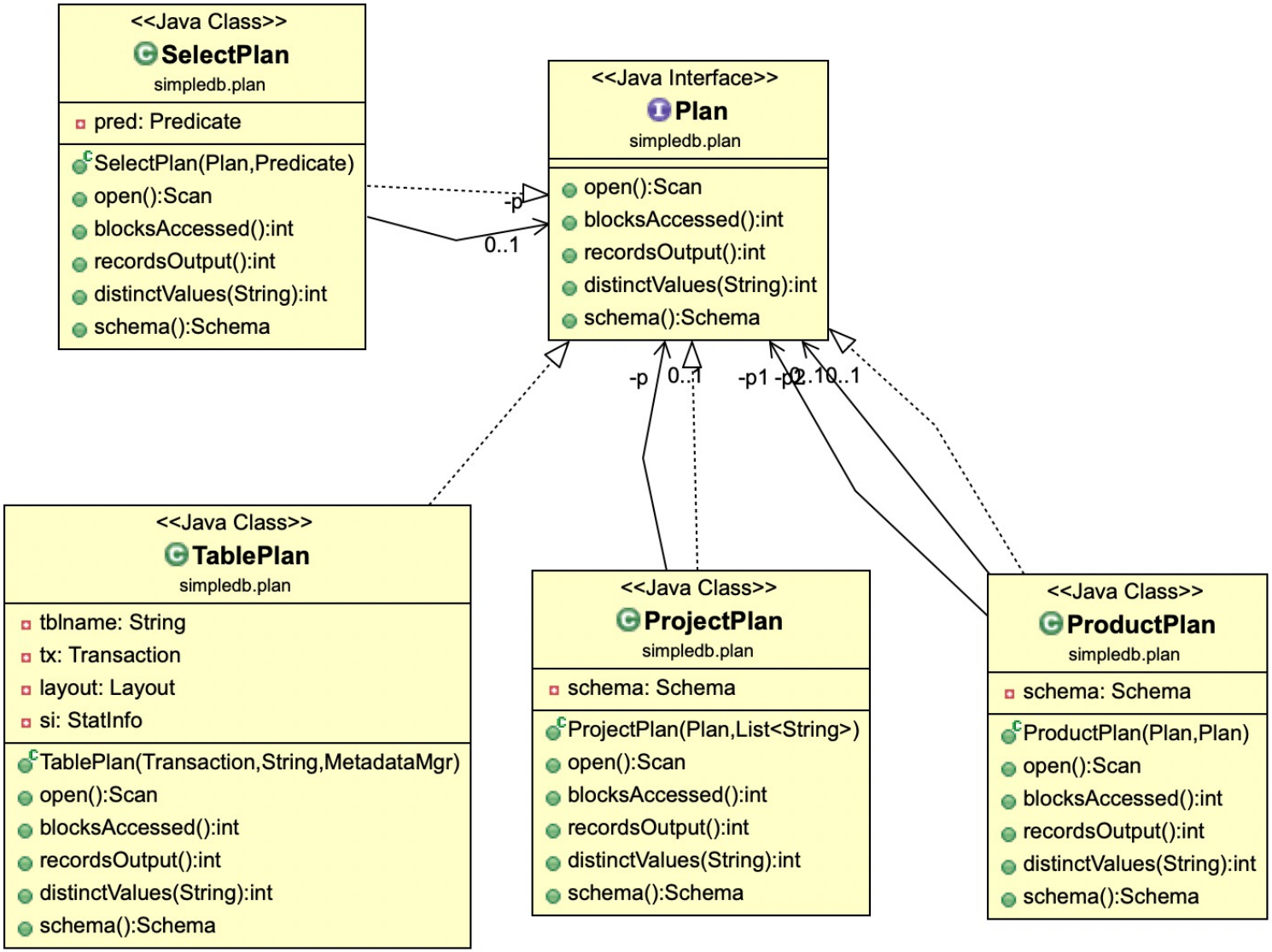
Plans: example

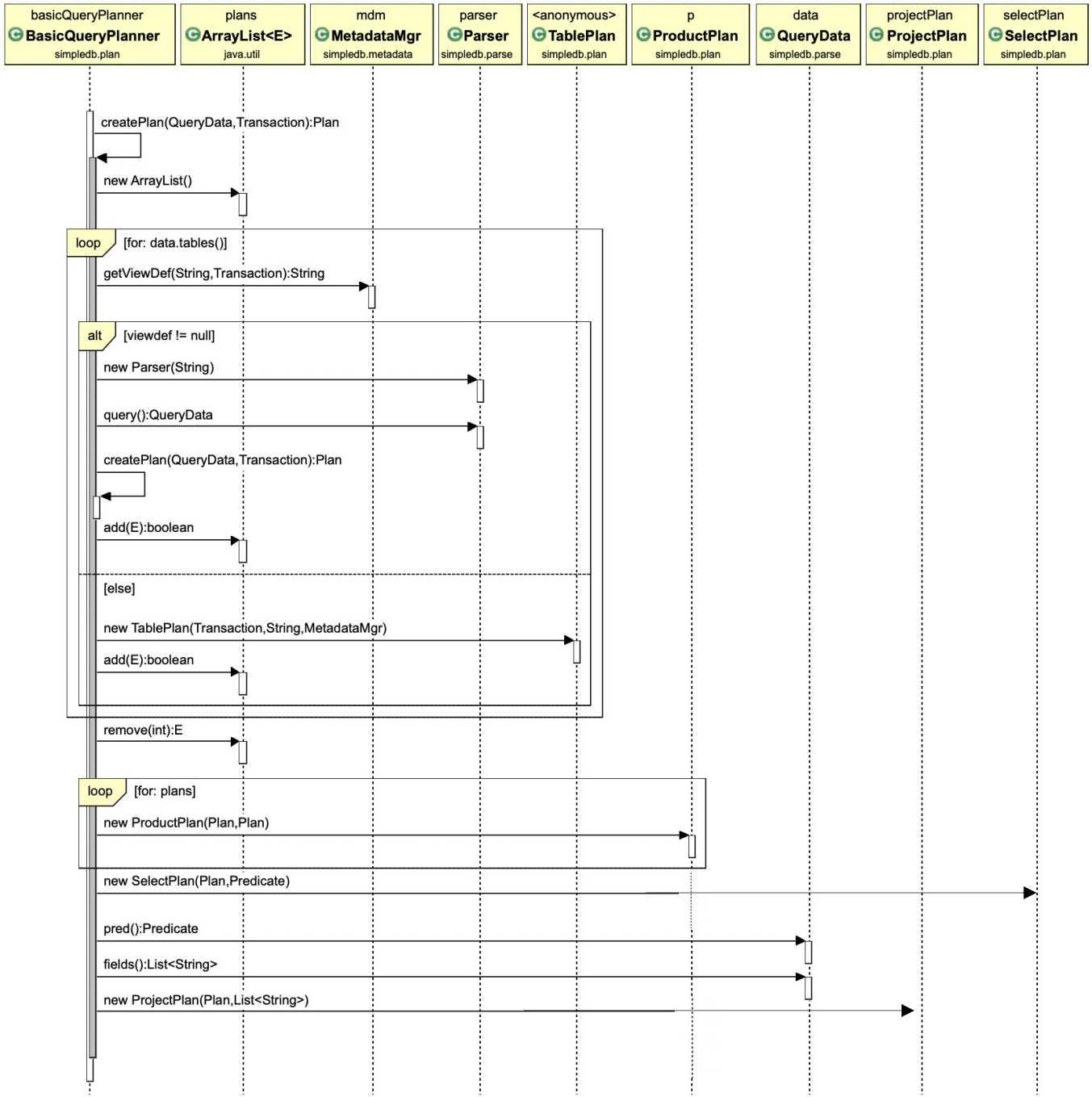
```
select SName
from STUDENT, ENROLL, SECTION
where SId = StudentId
and SectionId = SectId
and Grade = 'A'
and Prof = 'einstein'
```

(a)



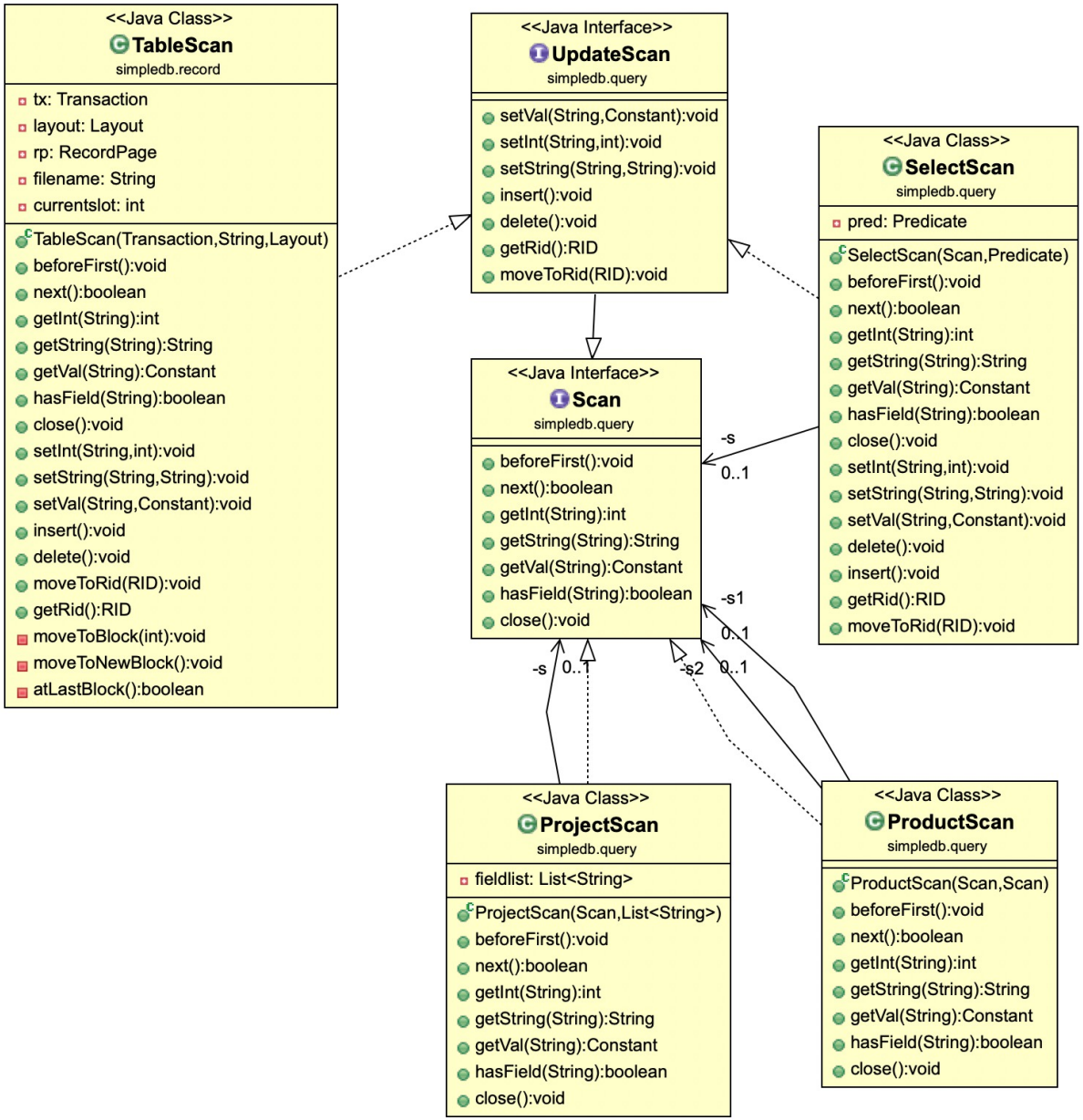
(b)


<<Java Class>>
SelectPlan
simpledb.plan
pred: Predicate
SelectPlan(Plan,Predicate)
open():Scan
blocksAccessed():int
recordsOutput():int
distinctValues(String):int
schema():Schema
-p
0..1
<<Java Interface>>
Plan
simpledb.plan
open():Scan
blocksAccessed():int
recordsOutput():int
distinctValues(String):int
schema():Schema
-p
0..1
-p1
-p2
2..10..1
<<Java Class>>
TablePlan
simpledb.plan
tblname: String
tx: Transaction
layout: Layout
si: StatInfo
TablePlan(Transaction,String,MetadataMgr)
open():Scan
blocksAccessed():int
recordsOutput():int
distinctValues(String):int
schema():Schema
<<Java Class>>
ProjectPlan
simpledb.plan
schema: Schema
ProjectPlan(Plan,List<String>)
open():Scan
blocksAccessed():int
recordsOutput():int
distinctValues(String):int
schema():Schema
<<Java Class>>
ProductPlan
simpledb.plan
schema: Schema
ProductPlan(Plan,Plan)
open():Scan
blocksAccessed():int
recordsOutput():int
distinctValues(String):int
schema():Schema


basicQueryPlanner
BasicQueryPlanner
simpledb.plan
plans
ArrayList<E>
java.util
mdm
MetadataMgr
simpledb.metadata
parser
Parser
simpledb.parse
<anonymous>
TablePlan
simpledb.plan
p
ProductPlan
simpledb.plan
data
QueryData
simpledb.parse
projectPlan
ProjectPlan
simpledb.plan
selectPlan
SelectPlan
simpledb.plan
createPlan(QueryData,Transaction):Plan
new ArrayList()
loop
[for: data.tables()]
getViewDef(String,Transaction):String
alt
[viewdef != null]
new Parser(String)
query():QueryData
createPlan(QueryData,Transaction):Plan
add(E):boolean
[else]
new TablePlan(Transaction,String,MetadataMgr)
add(E):boolean
remove(int):E
loop
[for: plans]
new ProductPlan(Plan,Plan)
new SelectPlan(Plan,Predicate)
pred():Predicate
fields():List<String>
new ProjectPlan(Plan,List<String>)

Query Manager

- Il Query Manager è in grado di eseguire le operazioni del plan
- Un'interrogazione è gestita da uno "scan":
  - Scan è un'interfaccia
  - Ci sono implementazioni per ciascun operatore e per la scansione di tabelle:
    - ogni nodo dell'albero che rappresenta un'interrogazione è un oggetto scan


<<Java Class>>
TableScan
simpledb.record
tx: Transaction
layout: Layout
rp: RecordPage
filename: String
currentslot: int
TableScan(Transaction,String,Layout)
beforeFirst():void
next():boolean
getInt(String):int
getString(String):String
getVal(String):Constant
hasField(String):boolean
close():void
setInt(String,int):void
setString(String,String):void
setVal(String,Constant):void
insert():void
delete():void
moveToRid(RID):void
getRid():RID
moveToBlock(int):void
moveToNewBlock():void
atLastBlock():boolean
<<Java Interface>>
UpdateScan
simpledb.query
setVal(String,Constant):void
setInt(String,int):void
setString(String,String):void
insert():void
delete():void
getRid():RID
moveToRid(RID):void
<<Java Class>>
SelectScan
simpledb.query
pred: Predicate
SelectScan(Scan,Predicate)
beforeFirst():void
next():boolean
getInt(String):int
getString(String):String
getVal(String):Constant
hasField(String):boolean
close():void
setInt(String,int):void
setString(String,String):void
setVal(String,Constant):void
delete():void
insert():void
getRid():RID
moveToRid(RID):void
<<Java Interface>>
Scan
simpledb.query
beforeFirst():void
next():boolean
getInt(String):int
getString(String):String
getVal(String):Constant
hasField(String):boolean
close():void
-s
0..1
-s1
0..1
-s
0..1
-s2
0..1
<<Java Class>>
ProjectScan
simpledb.query
fieldlist: List<String>
ProjectScan(Scan,List<String>)
beforeFirst():void
next():boolean
getInt(String):int
getString(String):String
getVal(String):Constant
hasField(String):boolean
close():void
<<Java Class>>
ProductScan
simpledb.query
ProductScan(Scan,Scan)
beforeFirst():void
next():boolean
getInt(String):int
getString(String):String
getVal(String):Constant
hasField(String):boolean
close():void